

# BOOTCAMP MINI 공시 데이터 파이프라인 설계서

검토 기준일 2026 년 10 월 2 일

## 1 설계 목적과 검토 범위

이 파이프라인은 기업 공시 원문을 추적 가능한 검색 자료로 바꾸는 데이터 기반이다. XML 의 문자와 표 구조를 보관하고, 수치 출현은 CSV 로 정리하며, 설명 문단과 정성 표 행은 BGE-M3 로 임베딩하여 Qdrant 에 저장한다. 검색 결과에서 원래 기업·기간·문단으로 되돌아갈 수 있도록 식별자와 위치를 유지한다.

검토 대상은 BOOTCAMP\_MINI 의 dart\_pipeline 코드, 게시된 data/processed, OpenDART 원본, 실행 중인 Qdrant 이다. 현재 구현은 검색 근거를 반환하는 단계까지이며, 이 폴더 안에서 LLM 답변을 생성하거나 재무 계정을 완전히 정규화하지 않는다. BOOTCAMP\_MINI\_PROTO 의 화면·Gemini 분석·KRX 수집은 별도 소비 서비스로 구분한다.

| 항목       | 현재 범위                                                                    |
|----------|--------------------------------------------------------------------------|
| 대상 기업    | 삼성전자 005930, SK 하이닉스 000660, 한미반도체 042700, 주성엔지니어링 036930, 원익 IPS 240810 |
| 보고 대상 기간 | 2023-2025 년. 매년 1 분기·반기·3 분기·사업보고서                                       |
| 원문 구성    | 정기보고서 60 건에서 본문 XML 60 개와 첨부 XML 26 개를 처리                                |
| 자료 규모    | 전체 블록 1,327,052 개, 검색 단위 52,674 개, 청크 54,204 개                           |
| 임베딩과 DB  | 고유 임베딩 입력 28,484 개, 1,024 차원 벡터, Qdrant 포인트 54,204 개                     |

## 전체 흐름

OpenDART ZIP 수집 → 인코딩 확인과 최소 XML 보정 → 엄격한 파싱과 문자 검사 → 원문·표·수치 보관 → 검색 단위 선별 → 겹침 없는 청킹 → 중복 입력 제거 → Intel GPU 임베딩 → Qdrant 업로드 → 검증된 파일 게시 → 기업·기간을 제한한 검색.

현재 파일과 벡터의 주요 연결은 정상이며 13 개 단위 테스트도 통과했다. 다만 게시 폴더의 audit/summary.json 등 일부 감사 파일이 빠져 있어, 표준 검증·재임베딩 명령의 실행 준비 상태는 완전하지 않다. 상세 상태와 개선 사항은 뒤에서 구분한다.

## 2 구성 모듈과 공시 수집

| 모듈                       | 책임                                                  |
|--------------------------|-----------------------------------------------------|
| config.py                | 프로젝트 위치, 5 개 기업, 3 개년, .env 읽기                      |
| collect.py               | 기업 코드·공시 목록·원문 ZIP·연간 재무 JSON 수집                    |
| preprocess.py            | 최소 XML 보정, 엄격한 파싱, 블록·표·수치·청크 함수                    |
| prepare.py               | 원문에서 source-structured-retrieval-audit 작업 자료 생성     |
| verify.py                | 원본 해시, 청크 재조립, 문단 연결, CSV 행 수 검증                    |
| embed.py                 | Intel GPU 임베딩, 벡터 캐시, Qdrant 업로드와 별칭 연결             |
| search.py                | 질의 임베딩, 기업·보고기간 필터, 원문 근거 반환                        |
| inspect.py               | 초기 탐색·진단용 코드. 현재 주 파이프라인은 prepare 이며 decode 함수만 재사용 |
| scripts/publish_data.ps1 | 검증된 작업 폴더를 게시 폴더로 교체                                |

### 수집 대상 결정

corpCode.xml 로 종목 코드와 DART 기업 고유번호를 연결한다. list.json 은 2023 년 1 월 1 일부터 실행일까지 정기공시를 조회한다. 보고서 이름에서 사업·반기·분기보고서와 보고 대상 연월을 추출하여 03-06-09-12 월만 선택한다. 접수 연도가 아니라 보고 대상 기간이 2023-2025 년에 해당하는지가 기준이다.

동일 기업·보고기간 후보가 여러 개이면 접수번호가 가장 큰 자료를 선택한다. 따라서 후속 정정본을 반영할 수 있지만, 정정 전후의 모든 버전을 보관하는 이력 저장소는 아니다. 현재 설정은 고정된 3 개년이며 자동으로 다음 해로 이동하지 않는다.

### 원문과 재무 API 의 역할

document.xml 은 이름과 달리 ZIP 바이너리를 반환한다[1]. ZIP 안의 모든 XML 을 처리하고 접수번호와 같은 파일명을 본문으로 판별하며 나머지는 첨부로 구분한다. fnlIttSinglAcntAll.json 은 각 연도의 사업보고서 코드 11011 로 수집하고 연결 CFS 가 없을 때 별도 OFS 를 요청한다[2]. 이 JSON 은 보조 재무 원본으로 남으며 prepare 의 수치 CSV 를 만드는 입력으로 사용하지 않는다.

기존 ZIP·재무 JSON 은 파일이 있으면 다시 받지 않는다. 네트워크 요청은 최대 4 회, 90 초 제한, 실패 시 1·2·4 초 대기로 재시도하며 정상 응답 뒤에는 0.2 초 쉼다. JSON 은 임시 파일 교체로 저장하지만 원문 ZIP 저장은 같은 수준의 원자적 교체를 적용하지 않는다.

### 3 XML 읽기와 문자 보존

#### 인코딩을 먼저 확정한다

ZIP 무결성을 확인한 뒤 XML 을 UTF-8 BOM 포함 형식 또는 EUC-KR 로 해독한다. 두 방식 모두 실패하면 예외로 중단한다. 읽을 수 없는 문자를 임의의 대체 문자로 바꾸거나 삭제하여 진행하지 않는다. 원본 ZIP 과 XML 바이트의 SHA256 은 수정하지 않은 원본을 기준으로 남긴다.

#### 확인 가능한 형식 오류만 보정한다

sanitize 는 주석·CDATA·처리 지시문과 정상 태그를 구분한다. 본문의 잘못 쓰인 &는 &amp;로, XML 기본 엔티티가 아닌 알려진 HTML 문자 참조는 숫자 문자 참조로 변환한다. 관측된 "<ACI 세미나>"와 같은 한글 문구는 태그가 아니라 눈에 보이는 문자로 보존한다. 속성 안의 문자를 보정할 때도 태그 이름과 경계를 유지한다.

예를 들어 "연구 & 개발"은 XML 파일에서는 "연구 &amp; 개발"로 보정되지만 추출된 본문은 "연구 & 개발"이다. 보정은 내용 요약이나 표현 수정이 아니라 파서가 읽을 수 있도록 만드는 최소 형식 변경이다. 변경 전후 문자열, 원본 문자 위치, 줄 번호와 사유를 기록하도록 구현되어 있다.

#### 엄격한 파싱과 이중 대조

보정 XML 은 ElementTree 로 엄격하게 파싱한다. 트리를 만들지 않는 독립적인 문자 추출 결과와 트리의 전체 텍스트를 공백 제외 문자 순서로 대조한다. 이어서 생성된 블록의 전체 텍스트와 같은 기준으로 다시 비교한다. 일치하지 않으면 일부 누락을 허용하지 않고 빌드를 중단한다.

DTD·지원하지 않는 선언·달하지 않은 주석 또는 CDATA·엄격한 파서가 거부하는 문자는 격리 대상으로 처리한다. prepare 는 실패한 원본을 작업 폴더 audit/quarantine 에 저장하고 오류를 올린다. 정상 처리된 보정 XML 은 audit/repaired\_xml 에 보관한다.

#### 보존 검사의 의미와 한계

검사가 보장하는 범위는 보정 XML 의 문자 데이터 순서이다. 원본 파일의 공백·줄바꿈·태그를 그대로 재현한다는 뜻이 아니며, 금융 용어나 표 헤더의 의미가 올바르게 해석되었음을 보장하지도 않는다. 이미지 내부의 문자는 OCR 하지 않는다. 406 개 이미지 참조는 문서

메타데이터에 기록되므로 XML 문자 추출 성공을 문서의 모든 시각 정보 추출 성공으로 표현해서는 안 된다.

현재 메타데이터에는 XML 86 개 모두 문자 보존 검사 통과와 총 9,339 건 보정이 기록되어 있다. 이번 검토에서는 원본 해시와 현행 자료 연결을 다시 확인했지만 XML 전체를 재추출하지 않았다. 보정 건수와 문자 검사 통과는 기존 실행 기록이라는 점을 구분한다.

## 4 산출물 구조와 출처 연결

data/opardart 는 수집 원본, data/processed 는 게시된 결과, data/processed\_build 는 다음 세대를 만드는 작업 폴더이다. 게시된 결과는 source-structured-retrieval-embeddings-audit 로 역할을 나눈다. CSV 의 행 수와 JSONL 의 행 수는 서로 다른 단위이므로 같은 수가 나와야 하는 것은 아니다.

| processed 아래 위치                  | 내용과 연결 기준                                                     |
|----------------------------------|---------------------------------------------------------------|
| source/documents.json            | 문서별 기업·종목 코드·연도·보고기간·접수번호·본문·첨부·ZIP/XML 경로·원본 해시·검사 결과        |
| source/documents/*.txt           | 태그를 제거한 전체 문자 내용                                              |
| source/blocks/*.jsonl            | 제목·문단·셀의 원문, paragraph_id 와 순서, xml_path, section, heading    |
| source/tables/*.jsonl            | 표 ID, 행·셀 병합 좌표, 상위 행에서 이어지는 항목명, 헤더 후보                       |
| structured/numbers.csv           | 숫자 출현과 문맥. document_id·paragraph_id·table_id·문자 위치로 원문 연결     |
| structured/table_cells.csv       | 셀 원문과 row_index·column·rowspan·colspan·셀 XML 경로               |
| retrieval/units.jsonl            | 검색 대상으로 선별된 문단 또는 표 행. paragraph_ids 로 원문 연결                  |
| retrieval/chunks.jsonl           | 검색 단위를 나눈 청크, 위치·순서·앞뒤 ID·임베딩 입력과 내용 해시                       |
| retrieval/embedding_inputs.jsonl | 동일 임베딩 입력을 제거한 내용 해시·텍스트 목록                                   |
| embeddings/                      | vectors.npy, vector_index.jsonl, progress.json, manifest.json |
| audit/ 및 examples.json           | 보정·분류·집계·검증 자료와 실제 유형별 예시                                     |

### 식별자의 계층

document\_id 는 종목 코드·접수번호·ZIP 내부 XML 순번으로 만든다. 예시는 005930\_20230515002335\_000 이다. 문단 ID 는 여기에 :p000632 를, 청크 ID 는 다시 :c0001 을 붙인다. 표 행 단위는 :t8:r00000 과 같이 표와 행 순서를 표시한다. Qdrant 포인트 ID 는 chunk\_id 에서 UUID5 로 결정되므로 재시도해도 같은 출처 포인트를 갱신한다.

숫자의 문자 위치는 해당 원문 블록 기준이다. 일반 청크의 문자 위치는 검색 단위 기준이며, 표 청크는 셀 원문 자체가 아니라 연결된 표 행 텍스트 기준이다. 표를 강조하려면 표 ID 와 행·셀

좌표로 되돌아가야 한다. 전체 XML 바이트 위치와 청크 문자 위치를 같은 값으로 취급하지 않는다.

## 5 수치 구조화와 검색 대상 선별

### 수치는 확정된 재무 사실이 아니라 출현 기록이다

numbers 는 원문 블록의 숫자 문자열을 찾고 원문 값·Decimal 문자열·명시된 단위·주변 문맥·위치를 CSV 에 남긴다. 날짜나 범위, 영문 제품 식별자, 수량 후보를 규칙으로 표시하지만 이는 후보 분류이다. △·▲ 부호와 괄호는 검토 표시를 남기며 음수 여부를 추측하지 않는다. 단위 배율과 숫자의 실제 귀속 기간도 자동 확정하지 않는다.

현재 예시의 “해외 9 개 지역총괄”에서 9 와 단위 개가 저장된다. 이는 조직 수량에 관한 출현이며 재무 계정은 아니다. 또 삼성전자의 “2023 년 1 분기 매출은 63 조 7,454 억원...” 문장은 숫자 CSV 에도 기록되지만 원문 설명과 함께 검색 대상에도 남는다. 수치 보관과 정성 검색을 서로 배타적으로 나누지 않는 구조이다.

### 문단과 표 행의 규칙

일반 문단은 한글·영문 글자 수가 12 개 이상이고 전체 길이가 25 자 이상이며 설명 표현이 있거나 한글·영문 글자 수가 40 개 이상이면 검색 후보가 된다. 제목과 매우 짧은 단편은 원문에 보관하되 별도로 임베딩하지 않는다. 이는 학습된 의미 분류기가 아니라 규칙이며 재현율과 정밀도를 별도 평가하지 않았다.

| 유형                           | 판정과 실제 예시                                                         |
|------------------------------|-------------------------------------------------------------------|
| narrative<br>20,071 개        | 수량 후보가 없는 설명 문단. 주요 사업·제품·위험에 관한 자연어                              |
| mixed_narrative<br>3,766 개   | 설명 문단에 수량 후보가 함께 존재. 매출과 증감률을 설명하는 본문도 보존                         |
| table_narrative<br>20,712 개  | 표 셀 하나라도 설명 문단 기준 충족. “각 부문별 매출액은 부문 등 간 내부거래를 포함...” 같은 주석       |
| table_qualitative<br>8,125 개 | 설명 문장 기준은 못 채워도 숫자 없는 두 셀 이상, 충분한 한글·영문을 가진 행. 유통업체·판매 채널·계약 조건 등 |

표는 ROWSPAN·COLSPAN 을 이용해 행·열 좌표를 복원한다. 이전 행에서 이어진 글자 항목은 carried\_labels 로 보관하고 셀 값은 중복 생성하지 않는다. 정성 검색용 행은 이어진 항목명과 셀

텍스트를 원래 순서로 " | "로 연결한다. 숫자 위주의 매출액·금액 행은 구조화 자료만 보관하며 LLM 으로 설명문을 만들어 임베딩하지 않는다.

표의 앞부분 최대 3 개 행에서 header\_candidates 를 찾지만 확정된 헤더나 회계 계정으로 취급하지 않는다. table\_cells.csv 의 셀을 숫자 CSV 의 셀 블록에 직접 연결하려면 document\_id 와 cell\_path/XML 경로를 대조해야 한다. 한 셀과 한 문단이 항상 일대일 대응한다는 전제는 없다.

## 6 청킹과 임베딩 입력

### 문서와 문단 경계를 먼저 유지한다

기업·보고기간·XML 문서별로 자료를 읽은 뒤, 일반 문단 하나 또는 표 행 하나를 검색 단위로 만든다. 인접 문단을 합치지 않으며 서로 다른 기업·연도·분기의 내용을 같은 청크에 넣지 않는다. 표에는 여러 원문 셀 블록의 paragraph\_ids 가 들어갈 수 있다.

split\_chunks 는 검색 단위를 기본 1,000 글자씩 고정 분할한다. 겹침은 0 이므로 청크를 순서대로 이어 붙이면 선별된 원문 단위가 정확히 복원되어야 한다. 청크에는 chunk\_order·chunk\_count·start\_char·end\_char·previous\_chunk\_id·next\_chunk\_id 를 남긴다. 문자 수 기준이므로 문장 중간에서 잘릴 수 있다.

| 설정       | 값과 선택 의도                                       |
|----------|------------------------------------------------|
| 청크 본문 길이 | 최대 1,000 글자. 기존 요청에 따른 간단하고 재현 가능한 기준          |
| 겹침       | 0 글자. 중복 본문을 방지하고 재조립 검사를 단순화                  |
| 입력 문맥    | section 최대 120 글자와 heading 최대 160 글자를 청크 앞에 추가 |
| 기업·기간    | 본문 앞에 역지로 붙이지 않고 Qdrant 필터용 메타데이터로 유지          |
| 중복 기준    | 제목 문맥을 포함한 embedding_text 의 SHA256             |
| 본문 출처    | 중복 벡터를 공유해도 출처별 chunk_id 와 Qdrant 포인트는 유지      |

### 1,000 글자와 토큰 제한은 다르다

BGE-M3 는 최대 8,192 토큰을 처리하는 모델이다[3]. 1,000 글자는 그 한도 자체가 아니라 출처를 작게 유지하고 메모리·검색 범위를 관리하기 위한 현재 기본값이다. 임베딩 실행 전에 실제 토큰나이저로 제목 포함 입력을 검사하며 8,192 토큰을 넘으면 조용히 자르지 않고 중단한다. 현재 manifest 에는 최대 943 토큰, 잘린 입력 0 개가 기록되어 있다.

### 중복 제거의 범위

54,204 개의 출처 청크가 28,484 개의 고유 임베딩 입력으로 연결된다. 동일 입력의 벡터 계산을 한 번으로 줄이되 반복된 공시의 출처는 삭제하지 않는다. 이 중복 제거는 의미가 비슷한 문장의 통합이나 공시 사건 수의 집계가 아니다. 같은 문장이라도 제목 문맥이 다르면 별도의 입력 해시가 된다.

1,000 글자와 겹침 0 이 검색에 최적인지는 검증하지 않았다. 긴 문단의 끝과 다음 청크의 시작을 함께 읽어야 할 경우 앞뒤 청크 ID 로 후속 확장을 할 수 있지만, 현재 search.py 는 그 확장을 자동 수행하지 않는다.

## 7 Intel GPU 임베딩과 벡터 저장

embed.py 는 로컬 BOOTCAMP/Models/bge-m3 의 SentenceTransformer 모델을 Intel XPU 에 올린다. 외부 LLM 을 호출하거나 모델을 재학습하지 않는다. Intel GPU 가 없으면 오류로 중단하며 자동 CPU 대체를 하지 않는다. 이 정책은 별도 프로토타입 소비 코드의 CPU 대체 정책과 구분해야 한다.

| 항목     | 현재 구현 또는 기록                                         |
|--------|-----------------------------------------------------|
| 모델     | BAAI/bge-m3. dense 표현만 사용, sparse-ColBERT 미사용       |
| 장치     | manifest 기록: Intel Arc 130V GPU 16GB                |
| 계산·저장  | GPU 추론 FP16, vectors.npy 저장 float32                 |
| 출력     | 1,024 차원. 로컬 모델 구성의 CLS pooling 과 L2 정규화            |
| 배치     | 기본 16. GPU 메모리 부족 시 8·4·2·1 로 감소                    |
| 입력 순서  | 토큰 길이와 내용 해시로 정렬해 패딩 낭비 감소                          |
| 완료 기록  | 고유 벡터 28,484 개, Qdrant 포인트 54,204 개, completed=true |
| 재개 지점  | progress.json 의 embedded-uploaded 와 identity        |
| 모델 동일성 | 모델의 safetensors.json-model 파일 해시와 데이터 해시 대조         |

### 캐시를 먼저 저장하고 그다음 DB 에 올린다

배치 벡터는 numpy 메모리 매핑 배열에 기록하고 flush 한 뒤 임베딩 진행 위치를 저장한다. 내용 해시와 벡터 행 번호는 vector\_index.jsonl 로 연결한다. 벡터를 재계산할 필요가 없는 출처 청크는 같은 벡터 행을 재사용한다. Qdrant 업로드는 256 포인트씩 수행하고 최대 4 회 재시도한다.

중단 후 동일한 작업 폴더와 설정으로 embed 를 다시 실행하면 저장 지점부터 진행한다. 데이터·모델 해시·모델 경로·차원·정규화·정밀도·torch 버전이 다르면 다른 자료와 섞지 않고 중단한다. 상태 JSON 은 임시 파일 교체로 저장하며 Windows 파일 잠금에 대해 최대 10 회 재시도한다.

## 실행 환경의 기록과 재현성

기존 README 에는 Python 3.11.15, torch 2.14.1+xpu, sentence-transformers 5.7.0, transformers 4.57.6, qdrant-client 1.19.1 이 기록되어 있다. 현재 manifest 에는 torch 버전과 장치 등이 들어 있지만 모든 라이브러리·드라이버 버전과 임베딩 처리 시간은 없다. requirements-embedding.txt 의 지정 버전과 실제 BOOTCAMP 환경이 같은지는 재실행 전에 별도로 확인해야 한다.

이번 검토에서는 모델 전체를 다시 실행하지 않았다. 저장된 벡터의 형태·유한 값·정규화 상태를 확인했다. 이것은 저장 형식 검증이며 검색 내용의 관련성이 높다는 성능 검증은 아니다.

## 8 Qdrant 구성과 검색 과정

Qdrant 는 127.0.0.1:6333 에 연결하며 데이터·모델 해시로 구분한 컬렉션을 사용한다. 현재 컬렉션은 snapdart\_bge\_m3\_4959f8bcd5\_07e49d0d 이다. 벡터 설정은 크기 1,024, Cosine 거리, on\_disk=true 이며 로컬에서 상태 green 과 정확한 포인트 수 54,204 개를 확인했다.

payload 는 embedding\_text 를 제외한 청크 메타데이터와 본문, 데이터·모델 해시를 담는다. stock\_code·year·period·document\_id·document\_role·content\_type·section 에는 keyword 인덱스를 만든다. 업로드 수가 청크 수와 같을 때 snapdart\_current 별칭을 연결한다[4]. 현재 search.py 는 이 별칭이 아니라 manifest 의 실제 컬렉션 이름을 조회한다.

### 삼성전자 특정 기간에서 설비투자를 찾는 예

질의를 “설비투자과 생산능력”으로 하고 stock\_code=005930, period=2023-03 을 지정한다. search.py 는 manifest 에서 모델 경로와 컬렉션을 읽고 질의도 같은 BGE-M3 로 임베딩한다. Qdrant 는 해당 기업·기간의 후보 중 벡터 유사도가 높은 자료를 기본 5 개 반환한다. 특정 단어의 정확한 포함 여부를 검색하는 방식은 아니다.

반환 자료는 score, chunk, source, source\_blocks\_file, source\_tables\_file 이다. chunk 에는 원문 발췌와 출처 식별자가, source 에는 문서 메타데이터가 들어간다. 문단 본문 전체를 API 에서 추가로 읽거나 CSV 숫자를 결합해 주는 기능은 아직 없다. 반환된 파일 경로와 ID 를 소비자 서비스가 이어서 사용한다.

### 현재 실제 출처 연결 예

examples.json 의 삼성전자 혼합 문단은 문서 005930\_20230515002335\_000, 문단 p000632, 청크 c0001 로 연결된다. 그 내용은 2023 년 1 분기 매출과 전년 동기 대비 변화 설명이다. 이 예시는 저장된 자료 구조를 보여주는 것이며 위 설비투자 질의에서 실제 상위 결과로 반환된다고 주장하는 것은 아니다.

원문은 source/blocks/005930\_20230515002335\_000.jsonl 에서 paragraph\_id 로 찾는다. 수치는 structured/numbers.csv 에서 같은 문서·문단 ID 로 대조한다. 표 결과라면 table\_id 와 row\_index 를 source/tables 에서 확인하여 행·열·병합 관계를 복원한다. financial\_2023.json 은 별도의 연간 재무 원본이며 정성 검색 결과에 자동 결합되지 않는다.

## 검색 범위와 성능의 제한

현재 search.py 는 기업과 선택적인 period 만 제한한다. 본문/첨부, 사업의 내용, 콘텐츠 유형 필터는 이 명령에 구현되지 않아 감사 첨부나 다른 섹션이 결과에 포함될 수 있다. BM25-동의어 사전-재순위 모델-유사도 임계값-출처 중복 제어도 없다. 기업-시기별 용어 차이는 임베딩으로 완화될 수 있으나 회계 의미의 동일성을 보장하지 않는다.

## 9 실행 순서와 게시 절차

아래 명령은 BOOTCAMP\_MINI 에서 실행한다. 원문 수집은 API 키가 필요하고, 임베딩·검색은 기존 Intel GPU 용 BOOTCAMP 가상환경과 로컬 모델, 실행 중인 qdrant\_mini 가 필요하다. 실제 재빌드는 이번 문서 작성 작업에서 수행하지 않았다.

```
$py = (Resolve-Path ..\BOOTCAMP\venv\Scripts\python.exe).Path
```

### 1 원문 수집

```
& $py -X utf8 -m dart_pipeline.collect
```

필요한 경우 .env 에 OPENDART\_API\_KEY 또는 DART\_API\_KEY 를 설정한다. 원문과 재무 JSON 은 존재하는 파일을 건너뛰므로 변경된 정정 자료나 오래된 재무 캐시를 어떻게 갱신할지 먼저 확인한다.

### 2 빈 작업 폴더에서 전처리와 검증

```
& $py -X utf8 -m dart_pipeline.prepare
```

```
& $py -X utf8 -m dart_pipeline.verify --data data/processed_build
```

prepare 의 기본 출력은 processed\_build 이며 비어 있지 않으면 중단한다. 실패한 작업 폴더를 정리할 때는 원본 opendart 와 게시 폴더 processed 를 보존한다. 다른 빈 폴더를 --output 으로 지정했다면 다음 단계의 --data 도 같은 위치를 지정해야 한다.

### 3 임베딩과 Qdrant 업로드

```
& $py -X utf8 -m dart_pipeline.embed --data data/processed_build --batch-size 16
```

임베딩이나 업로드가 중단되었다면 prepare 를 반복하지 않고 같은 embed 명령으로 재개한다. 캐시 동일성 검사에 실패하면 설정을 억지로 맞추지 말고 새 작업 폴더로 빌드한다.

### 4 검증 결과 게시와 검색

```
.\scripts\publish_data.ps1
```

```
& $py -X utf8 -m dart_pipeline.search "반도체 설비투자" --stock-code 005930 --period 2023-03 --top-k 3
```

게시 스크립트는 검증 파일·완료 manifest·포인트 수를 확인하고 processed 를 processed\_previous 로 옮긴 뒤 작업 폴더를 processed 로 바꾼다. 두 번째 이름 변경에 실패하면

이전 폴더를 되돌린다. 성공 시 이전 가공 폴더와 과거 inspection 을 삭제하지만 원본과 프로토타입은 보존한다. 경로와 재분석 지점을 점검하여 예상한 data 폴더 안에서만 이동·삭제한다.

Qdrant 별칭 교체는 embed 완료 시 먼저 일어나며 파일 게시와 하나의 트랜잭션으로 묶이지 않는다. 게시 시 잠시 파일 경로가 비거나 별칭 소비자와 파일 세대가 다를 수 있다. 서비스 실행 중 세대 교체는 점검 시간에 실시하고 같은 데이터 해시를 확인하는 운영 절차가 필요하다.

## 10 현재 검증 결과와 우선 개선 사항

2026 년 10 월 2 일에 코드·현재 파일을 직접 대조하였다. 원본 재수집, 전처리 재생성, 모델 재임베딩, 검색 품질 평가는 수행하지 않았다. 기존 기록과 이번에 다시 검증한 사실을 아래처럼 나눈다.

| 검증 항목     | 이번 확인 결과                                                         |
|-----------|------------------------------------------------------------------|
| 원본과 파일 연결 | 86 개 XML 의 원본 SHA256 일치. 모든 문서의 txt-blocks-tables 파일 존재          |
| 청크와 원문 단위 | 54,204 개 청크의 순서·문자 위치·내용 해시와 52,674 개 단위 재조립 확인. 최대 1,000 글자     |
| 원문 문단 연결  | 모든 검색 청크의 paragraph_ids 가 해당 문서의 블록에 존재                          |
| 구조화 행 수   | numbers.csv 747,233 행, table_cells.csv 1,463,408 행               |
| 벡터 캐시     | 28,484 × 1,024 float32. 유한 값이며 벡터 길이는 약 1                        |
| DB 현재 상태  | Qdrant green, 정확한 포인트 수 54,204 개, snapdart_current 가 현재 컬렉션을 가리킴 |
| 기존 실행 기록  | 문자 보존 통과, 9,339 건 보정, 최대 입력 943 토큰, 입력 잘림 0 건                    |
| 코드 테스트    | tests 의 13 개 단위 테스트 통과. XML·수치·규칙·표 병합·정정보 선택 검증                 |

### 우선순위 1 감사 파일 복구

현재 processed/audit 에는 summary.json·repairs.jsonl·routing.jsonl 이 없다. 원문과 벡터 검색 자료는 남아 있고 별도 대조도 통과했지만 verify.py 와 embed.py 는 summary.json 을 필수로 읽으므로 게시 폴더를 대상으로 그대로 실행하면 실패한다. 보정·선별 이유의 상세 추적도 제한된다. 같은 데이터 해시의 보관본이 있는지 확인한 뒤 복구하거나 원본부터 새 작업 세대를 만들어야 한다. 이번에는 기존 자료를 수정하지 않았다.

### 우선순위 2 게시 세대와 캐시 통제

파일과 Qdrant 별칭을 교체하기 전에 데이터·모델 해시를 함께 검사하고 소비자가 같은 세대를 읽도록 해야 한다. 현재 게시 스크립트의 개수 검증만으로 잘못 섞인 세대를 충분히 막을 수 없다. 이전 세대 보존과 롤백 정책, 원문 ZIP 의 임시 파일 저장·무결성 검사도 보완 대상이다.

### 우선순위 3 검색과 수치 해석의 평가

대표 질의와 정답 문단을 만들어 상위 검색 결과의 관련성·누락을 평가한다. 실제 수요가 확인되면 본문·섹션 필터, 용어 사전, 키워드와 벡터의 결합, 앞뒤 청크 확장 등을 추가한다. 수치 계산에는 financial JSON의 계정·기간·연결/별도·단위를 먼저 확인하고 XML의 숫자 출현을 검증 없이 사용하지 않는다. SQL 저장·자동 스케줄링·LLM 응답 생성은 이후 단계로 구분한다.

## 11 설계 원칙과 검토 근거

### 유지해야 할 원칙

원본은 보존하고 파생 자료만 다시 만든다. 파싱이나 검증에 실패한 결과를 정상 자료처럼 게시하지 않는다. 수치와 설명을 원문 위치로 연결하고, 확인되지 않은 금융 의미를 새로 만들어 넣지 않는다. 반복 계산은 줄이되 출처를 삭제하지 않는다. 처리 속도와 검색 품질, 문자 보존과 의미 해석을 서로 다른 검증 대상으로 다룬다.

### 설계서가 다루지 않는 범위

이 문서는 BOOTCAMP\_MINI 의 데이터 파이프라인 설계와 현황이다. 산업 인사이트 문장의 생성·평가, 화면의 주가 지수 선택, KRX API 인증과 일별 수집, 투자 판단의 타당성은 이 코드의 구현 범위가 아니다. 금융 서비스에 사용할 때에는 공시 정정보·실적 기간·전사와 사업 부문 범위를 소비 계층에서 별도로 확인해야 한다.

### 직접 검토한 프로젝트 자료

dart\_pipeline/config.py, collect.py, inspect.py, preprocess.py, prepare.py, verify.py, embed.py, search.py; scripts/publish\_data.ps1; README.md; requirements-embedding.txt; tests/test\_audit.py, test\_preprocess.py, test\_retrieval.py 를 확인하였다. 현재 source/documents.json, units-chunks-embedding\_inputs, 구조화 CSV, vectors-vector\_index-progress-manifest, examples.json 을 대조했다.

data/[RM\_20261001\_162104]processed/audit/summary.json 은 같은 데이터 해시의 보관 기록으로만 참고했다. 현재 감사 파일이 존재한다고 대신 간주하지 않았다. 검증 명령의 결과 파일을 덮어쓰지 않고 별도 읽기 검토를 수행했으며, 원본 자료·모델·Qdrant 컬렉션은 변경하지 않았다.

### 공식 기술 자료

[1] 금융감독원 OpenDART 공시서류원본파일 개발가이드. document.xml 의 ZIP 응답과 접수번호 기준 수집을 확인.

<https://opendart.fss.or.kr/guide/detail.do?apiGrpCd=DS001&apild=2019003>

[2] 금융감독원 OpenDART 단일회사 전체 재무제표 개발가이드. 사업연도·보고서 코드·연결/별도 구분을 확인.

<https://opendart.fss.or.kr/guide/detail.do?apiGrpCd=DS003&apild=2019020>

[3] BAAI BGE-M3 공식 모델 카드. 1,024 차원·8,192 토큰 및 dense 와 다른 표현 방식의 구분을 확인.  
<https://huggingface.co/BAAI/bge-m3>

[4] Qdrant 공식 포인트·인덱스 문서. 포인트 ID·벡터·payload 및 필터 인덱스 구조를 참고.  
<https://qdrant.tech/documentation/manage-data/points/>  
<https://qdrant.tech/documentation/manage-data/indexing/>

외부 기술 자료는 모델·API·DB 의 일반 명세 확인에 사용했다. 프로젝트의 실제 파라미터와 처리 상태는 코드와 현재 파일을 우선 근거로 삼았다.